

8-5 · 데이터 파이프라인 구축 및 최종 RAG 챗봇 개발 · 실습 1.5H

Airflow DAG + RAG 챗봇 통합 구축

최종 DAG 완성, 챗봇에 출처와 갱신 시각 표시

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① 전 구성 요소(compose·Airflow health·컬렉션)를 전수 점검한다
- ② 스케줄·재시도·내용 기반 uuid5 를 갖춘 rag_pipeline_final 을 쓴다
- ③ 챗봇 근거에 적재 시각을, 사이드바에 문서 수를 띄운다
- ④ 설정을 상단 한곳에 모으고 컬렉션 이름 세 곳을 대조한다

확인 포인트를 지날 때마다 다음 단계로 — 하나라도 안 되면 거기서 해결하고 넘어갑니다

여섯 단계 — DAG 폴더와 챗봇 폴더를 오가며

~/airflow-lab 의 dags 와 ~/rag-chatbot 의 app.py 를 함께 열어 둡니다

1. 전수 점검	2. 최종 DAG	3. 근거 표시	4. 사이드바	5.6. 설정·대조
compose ps health · collections	@daily · 재시도 uuid5	적재 시각 ingested_at	문서 수 근거 개수 슬라이더	os.getenv 컬렉션 이름 세 곳

1단계

전수 점검 — 셋 다 통과해야 출발

1

docker compose ps

전 서비스가 (healthy)

2

curl -s http://localhost:8080/health

metadatabase·scheduler 가 healthy

3

curl -s http://localhost:6333/collections

목록에 lecture_docs 가 보인다(8-3 적재분)

하나라도 안 되면

여기서 해결

후 진행

최종 DAG — 8-3 에서 달라진 여덟 줄

줄	뜻
INCOMING = BASE/incoming	새 문서 투입구(.txt)
glob(INCOMING/*.txt)	폴더의 파일을 읽어 raw.json
schedule="@daily"	매일 자동 갱신
retries=2	일시 장애는 자동 재시도
retry_delay 3분	timedelta(minutes=3)
uuid5(NAMESPACE_URL, t)	같은 문서는 한 건(먹등)
payload ingested_at	적재 시각 — 얼마나 낡았나
if not docs: return	빈 입력은 건너뛰고 정상 종료

collect >> transform >> embed_and_load — 웹 UI 에 rag_pipeline_final 이 뜨고 다음 실행 예정 시각·재시도 설정이 보이면 통과

챗봇 화면에 시각과 문서 수를

- 1 근거마다 적재 시각 — `st.caption`**
`h.payload.get('ingested_at')[:19]` — 오래된 자료인지 사용자가 판단
- 2 사이드바에 `st.metric("문서 수")`**
`get_collection(COLLECTION).points_count` — 차원은 `caption`
- 3 연결 실패는 `st.error` 로**
`try/except` — 「Qdrant에 연결하지 못했습니다」 + `type(e).__name__`
- 4 슬라이더 · 대화 지우기**
`st.slider 1~10 (기본 3)` · 버튼으로 `messages` 를 비우고 `st.rerun()`

DAG 돌린 뒤 새로고침

문서 수가
늘어난다

설정 한곳에, 이름 세 곳 대조

COLLECTION	os.getenv("COLLECTION", "lecture_docs")
QDRANT_HOST	os.getenv("QDRANT_HOST", "localhost")
모델·임계값	EMBED_MODEL · CHAT_MODEL · THRESHOLD 0.3
클라이언트	@st.cache_resource get_clients()
DAG 쪽	grep -n "lecture_docs" dags/rag_pipeline_final.py
챗봇 쪽	grep -n "lecture_docs\ COLLECTION" app.py head
실제 컬렉션	curl -s localhost:6333/collections python -m json.tool

세 곳의 이름이 완전히 같아야 한다 — 오타 한 글자로 'DAG 는 성공했는데 챗봇이 모른다'가 된다

이름 한 글자 — DAG 성공, 챗봇은 「모른다」

증상 · 원인

DAG 는 초록인데 챗봇이 새 문서 내용을 답하지 못한다

- DAG 와 챗봇이 다른 컬렉션 이름을 본다(오타 포함)
- 이전 DAG(rag_pipeline)를 트리거해 새 수집 로직이 안 돌아갔다
- 8-5 에서 가장 흔한 사고 원인

해결

양쪽 코드의 컬렉션 이름을 대조하고 collect 로그를 본다

VS

- grep 두 줄 + curl collections 로 세 곳 대조
- Grid 에서 rag_pipeline_final 의 collect 로그에 수집 건수 확인
- 이 대조를 습관으로

이 1.5시간이 끝날 때 확인할 것

- 1 docker compose ps 전부 (healthy), health 응답 둘 다 healthy
- 2 웹 UI 에 rag_pipeline_final · 다음 실행 예정 시각과 재시도 설정이 보인다
- 3 근거 보기에 적재 시각이, 사이드바에 문서 수가 뜬다
- 4 app.py 상단에 COLLECTION·QDRANT_HOST 등 설정이 모여 있다
- 5 DAG·app.py·실제 컬렉션 세 곳의 이름이 완전히 같다

이제 새 문서를 넣어 답변까지 이어지는지 — 다음 1H 에서 처음부터 끝까지

8-5 · 실습 1.5H 마무리

다음은 엔드투엔드 검증·시연 (1H)

새 문서를 투입구에 넣고 DAG 를 돌려 챗봇이 그 내용으로 답하는지 처음부터 끝까지 확인합니다

✓ 기준선 `points_count` 기록

✓ 「레이크하우스가 뭐야?」 전후 비교

✓ 두 번 돌려 먹등 · Qdrant 내렸다 올려 재시도

✓ `requirements.env.example` 로 패키징

가져갈 것

웹 UI 의 최종 DAG · 고친 `app.py`